Hey everyone,

I've gone through the full flow + team structure + sample site (AngelHack HackGlobal) and put together a clear proposal so we can start building tomorrow / day after.
This is **my suggested plan as System Architect** – Nikhil owns overall vision/strategy, and Harshita leads HR/ops/growth. I'll focus on the technical side and coordination within that.

Please read once and share feedback - we can refine together.

1. Goal & Quality Bar

We're building a **proper hackathon platform**, not a typical college project:

- Premium landing experience (visuals, copy, responsiveness).
- Smooth participant flow: **landing → signup/login → team → payment → dashboard → submission**.
- Solid admin experience for submissions, scoring, and leaderboard.

We **can use paid infra** (Supabase paid tier, email provider, etc.), so we'll optimize for developer speed + scalability instead of cutting infra corners. Nikhil – Are we allowed to use paid infra or we have to create the website using free software and limits?

2. Proposed Tech Stack (for feedback)

From a tech and scalability perspective, here's what I recommend:

Frontend / Full- stack Framework

- **Next.js (React + TypeScript)**
  - Modern standard for React apps, good SEO, supports mixed SSR/SSG, and ships with API routes so we don't need a separate backend repo.

Backend / Database / Storage / Auth

- **Supabase (Postgres + Auth + Storage)**
  - Managed Postgres with connection pooling, read replicas and production-ready scaling patterns, so it can comfortably handle large user bases when needed.

    - Supabase Auth for email/password + OAuth later; Storage for PPT/docs uploads.

Payments

- **Razorpay**
    - Use Razorpay's order + checkout + webhook flow via our Next.js API routes.
    - For now we'll **show fee per person / per team in USD only** for international friendliness (Razorpay supports 100+ currencies including USD when international payments are enabled).

Emails

- Any of: **Resend / SendGrid / Mailgun / SES**
    - For registration confirmations, team invites, submission confirmations, results.

Hosting

- **Vercel** for the Next.js app.
- **Supabase** as managed backend.

If we ever need SSO/enterprise identity later, we can plug in Auth0/Clerk on top; for this stage, Supabase Auth is simpler and enough.

This is my suggestion as System Architect - happy to adjust if Nikhil/others have strong preferences.

3. High- Level Architecture

Client (browser)

- Next.js routes for: Home, About, Tracks, Prizes, FAQs, Auth, Dashboard, Admin.
- Uses Supabase client for auth/DB, and Next.js API routes for payments/admin.

Next.js API Routes

- `/api/razorpay/create-order` → creates Razorpay order.
- `/api/razorpay/webhook` → verifies payment signature, updates `payments` and team/user status to `PAID`.[15][13]
- `/api/admin/...` → secure endpoints for admin (scores, leaderboard export etc.).

Supabase

- Postgres tables: `users`, `teams`, `team_members`, `payments`, `submissions`, `scores`.
- Auth: user accounts, sessions, row-level security.
- Storage: PPT/docs uploads for submissions.

Razorpay

- Payment gateway, with support for USD and other international currencies once international payments are enabled for the account.

4. Product Flow → Features

Short version of what we will build, mapped to the earlier flow:

- **Phase 1: Landing** – Home, About, Tracks, Prizes, FAQs, with "Register Now" and "View Tracks".
- **Phase 2: Registration** – Signup/login with email + password (Google later); profile fields (Name, Email, Phone, College/Company).
- **Phase 3: Team Formation** – Solo participation, or create/join team via invite code + email invites.
- **Phase 4: Payments** – Show fee per person/team **in USD**, integrate Razorpay for payment and webhook verification.
- **Phase 5: Confirmation & Dashboard** – Confirmation email, dashboard with track, team, payment and submission status.
- **Phase 6–7: Submission** – Resources page + submission form (title, description, track, repo, demo, PPT/docs upload).
- **Phase 8: Admin Review** – Admin dashboard with filters, scoring (innovation, technical, UI/UX, scalability) and comments.
- **Phase 9: Results** – Public leaderboard, winner emails, optional live event embed.

Status lifecycle we'll respect in the data model:

`REGISTERED → TEAM_CREATED → PAID → SUBMITTED → UNDER_REVIEW → SHORTLISTED → WINNER`.

5. Roles & Responsibilities (clarified, not hierarchical)

This is **not** me "managing" everyone – just clarifying who focuses on what, so we don't overlap or miss pieces.

Nikhil – Vision + Strategy + Control

- Owns overall vision of the hackathon and platform.
- Final call on scope/MVP and prioritization.
- Aligns product decisions with brand and growth.

Harshita – HR + Operations + Growth

- Leads ops, partnerships, and growth strategy.
- Coordinates announcements, outreach, participant communication.
- Works closely with Nikhil on positioning and with design/content to keep messaging on point.

Kishlaya – System Architect (its ME :))

- Proposes and documents tech stack & architecture (this doc).
- Designs data model and API contracts.
- Sets up repo, base Next.js + Supabase wiring, and coding standards.
- Coordinates between backend/frontend/design so the end- to- end flow works.
- Does code reviews + ensures quality, but not "manager" of Nikhil/Harshita – just owning the technical side.

Santhosh – Backend + Integrations

- Supabase schema, indices, RLS policies.
- Razorpay integration (order creation, webhook, mapping to payments).
- Admin APIs, email triggers, any additional backend workflows.

Smriti – UI/UX

- Figma designs for landing, dashboard, submission, admin.
- Design system: colors, fonts, spacing, components.

- Handoff to devs with responsive behavior + UX details.

Debojeet – Frontend + Content

- Implement Figma in Next.js (pages, components, layout).
- Hook up to Supabase/Auth/API endpoints.
- Owns marketing/content integration (tracks, prizes, FAQs, copy tweaks) + basic SEO.

6. Immediate Next Steps

If we're okay with this direction:

1. I'll:
    - Create the GitHub repo with base Next.js + TS + Supabase setup.
    - Draft the DB schema in a short doc/diagram and share.
2. Smriti can start with Figma (even wireframes) for landing and dashboard.
3. Debojeet can then begin implementing the landing shell + layout.
4. Santhosh can start setting up Supabase project + tables + Razorpay integration skeleton.

We can adjust tasks/timeboxes once we talk.

7. Feedback & Call

I'm **not available tonight**, but I'm free **tomorrow between 11 am – 1 pm IST**.

- Please **reply here** with:
    - Any concerns with this stack/architecture.
    - Any features you feel are missing or out of scope.
- If you're up for an informal call:
    - Drop a time that works for you between **11–1**, and we'll lock a 30–45 min sync to finalize and start execution.

This is just a proposal from my side as System Architect – open to feedback from Nikhil, Harshita, and everyone before we lock it in.

I need everyone's views before starting anything :)